What is a RESTful API？ ｜ Creating a REST API with Node.js

Welcome to a brand new series on this channel. In this series we'll have a look at how we create a RESTful API with Node.js. Let's dive right into it. So let's start building a RESTful API with Node.js and for that first of all let's have a look at what exactly a RESTful API is, what it's made up of and what we should keep in mind whilst we build it.

So what is a RESTful API actually? Well RESTful stands for Representational State Transfer and the whole idea behind a RESTful API or behind that name really is that we can use it to transfer data around. Now that sounds bigger than it maybe is. The general purpose of why we want to use it is that it's an alternative to a traditional web page for cases where that traditional web page just doesn't fit our needs.

Let's maybe have a closer look at this. So what is a RESTful API? Imagine we have a client which is our browser so we are the client and we have some server obviously like a web page. There we send a request for example if we enter something into the URL bar and we get back a response and for a normal web page as you know that's a couple of HTML pages for the different URLs we enter.

So we have a starting page, we have a products page, an order page, something like that. That is a traditional web page. Now if you're building a single page application it's going to be a bit different.

There you also send a request and you get back a response but you really only have this step only once because you get back one HTML page which contains a bunch of JavaScript in the end that will then dynamically re-render that page. But this is how web pages work. Now that's great for traditional web apps but what about some other cases where we also want to reach out to some server but where we don't really need HTML? Let's have a look at some of these cases.

We still got a server and by the way a server here also means that we probably have some database running on the server or running on another server but being connected to this server. And now let's have a look at different clients. Maybe a mobile app running on our smartphone and that mobile app obviously also needs to store and fetch data.

The problem just is it doesn't need HTML for that. The data is not transferred as a text file or something like this or as a HTML file I should say. Instead we use some other data format and we really are just interested in that data.

A similar case is if our client is some code, some application. Think of the Google Geolocation API where we can send coordinates and we get back a parsed address for example. That also is

a RESTful API just to give a little spoiler here and the idea here is that whilst we of course need to send data and get data we're not interested in a HTML page.

We just want to exchange data so that we can use it in our code. And finally I already mentioned it the single page application case. Here we actually have a web app but only for the first request we need HTML.

For all subsequent requests we only want to exchange data, send some data to the server, fetch some data from it and therefore here we also just need one HTML page and no more thereafter. You can't build all of that with a traditional server side setup because a RESTful API in the end is also just a normal server. The big difference in the end is that we don't care about this HTML stuff we just want to send data back and forth and that RESTful APIs are stateless backends.

They don't care about the individual client which connected to it. So if we have our client here and we have our RESTful server which is a normal server having some URLs it is able to accept requests on and so on then we might have these endpoints. So these are the URLs supported by our servers.

These URLs then each might also support different HTTP verbs. So we have GET and POST requests but we might also be able to support a DELETE request. For example here to delete a user or the same for POSTS maybe we have a PATCH request that's also HTTP verb which exists or for products maybe just a GET request.

So this is how a RESTful API could look like. We have a couple of URLs and each URL then possibly has a couple of different HTTP verbs and therefore types of requests it supports. And then from our client we can send a request.

If we're having a web app we would call this an AJAX request, asynchronous HTTP request. One where we don't send a request and get back a new page but one where we send a request our current page keeps on running and eventually we get back a response which we then if we're talking about a web application typically handle with JavaScript to re-render the DOM or do whatever we need to do with that response. And if we have some other kind of application like a mobile app then we still would have some tools provided by Java or Swift whatever it is whatever you use for writing that app that would be able to send a request and handle a response you get back.

This is the idea behind a RESTful API. We have a server with the URLs supporting different types of HTTP requests for the given URLs but all we do is we exchange data. Now if we have a look at this setup here this data is typically exchanged in JSON format.

It's not a must though you can send XML data, URL encoded data like where you have like query parameter style, form data. You're not limited to JSON. I'd say the one thing you typically always have is you're not sending HTML around though theoretically you could also do that and parse it

but not with the goal of rendering it through the browser because these API endpoints here are not going to get targeted directly by the browser in a sense of the user entering any of these URLs.

That's not going to happen. All these URLs are going to be targeted by background requests like a XML HTTP AJAX request sent from JavaScript or the equivalent for a mobile app. This is how we use it and we use it to exchange data because that is essentially the only way we can connect a single page application or a mobile app to some backend.

They don't want HTML. They want the data. They want to send data too.

This is the idea behind a RESTful API. Now if we go into theory land here then this is not necessarily a RESTful API. A RESTful API is a clearly defined construct and just having a couple of URLs with different supported HTTP endpoints is theoretically just a backend and an API you created.

We often call all these APIs RESTful though because it's the word or the expression for a backend that is not a traditional server sending back HTML but data but theoretically we got a couple of constraints that really turn an API into a RESTful API. We have six such constraints where one is optional and now this is really theoretical. We're also going to build one in this series so that it's a bit easier to grasp I guess.

The first constraint is a client server architecture. If we're building a RESTful API then we have a clear separation of concerns between our backend which is there to manage data, do calculations and send us back data and our frontend which could be a single page application or a mobile app which is responsible for the UI. Our RESTful API also is stateless so we don't store any client context like a session on it.

That's super important. A RESTful API doesn't care about the clients connecting to it. It doesn't care if it's reached by a single page application and a mobile app and maybe some other application.

It doesn't store anything which clearly binds it to one of these applications like a session. It's not handling sessions. It's not caring about sessions and this is going to become important when we add authentication.

We have cacheability in a sense that a RESTful API typically should also express itself or tell the client whether responses can be cached or not. This is also kind of the case if you don't explicitly set it up then there's just some default going to get used but you can't clearly define if for example for a GET request you want to cache the response, you want to allow the browser to cache the response and for how long that should be the case or if you would absolutely want to forbid any caching because you know that your data changes so frequently that caching doesn't make sense. You can set up caching responses here too to really make sure that the client is using the

API in an efficient way and this is just something you can do in a sense of you can clearly tell that caching should be enabled for example.

We also can build our RESTful API in some layered system which means the client connects to some server but that server doesn't necessarily have to be our final API. It could be some in-between server which forwards the requests or which sends back a response but behind the scenes also reaches out to our API and as I said we're really in theory land here but we just don't have the guarantee that our RESTful API or we don't give the guarantee I should say that our RESTful API is the final point in the traveler or in the journey of the request coming from the client. We also have a uniform interface which means that resources are identified in requests so we send a request to let's say slash users and we send a get request that clearly identifies one resource the users get resource and that the data we transfer can be decoupled from the database schema so if we store a user in a certain way in the server-side database we don't necessarily have to transfer it like this to the user we can deviate from that schema.

Also it's good if we have self-descriptive messages and links to further resources for example if we send a request to get users we would probably get back a list of all the users and then it would be really good if for each user object we don't just get let's say the id but we also get a link to which we would have to send a subsequent request to get the data for that user so that we don't have to guess about the API endpoint we would send a request to. Because that is something crucial to keep in mind of course when you're building an API you only have these addresses you only have these URLs and if you're not aware of them if you have no documentation to look it up and if the API doesn't send information about other addresses back in responses then you have no chance of using that API because how would you know to where you send a request. If you compare it to a web application a traditional what i mean where you have multiple html pages there the user would navigate around with links so there we also have that information about other pages we can visit and it's kind of the same here for the RESTful API it's good if we provide this information back to our clients and an optional request is code on demand that means that theoretically it would be allowed and still be a RESTful API if we implement it such that it gives the client back some executable code and that's not something we're going to build here so it really just means it doesn't have to be just data it could also be some code that the client can execute rather than just the data for that code and again these here these constraints are all just theory constructs so these are really just that's the theoretical definition of a RESTful API we're going to build one here and we're going to build one that works and that makes sense so don't learn that by heart so you don't need to know all of that be aware of that stateless thing it's really important of the clear separation between client and server that seem to be the most important things to me because that's something which often is hard to grasp and you often get asked well how can angular your angular single page application how can i create a session on the server and the answer is you can't really do that because in a single page application you use a RESTful API because you only need the data and a RESTful API shouldn't really care about the client connected to it shouldn't really manage sessions because

you never reload pages anyways so that's important to know you're independent from the client you're stateless now enough of the theory let's build a RESTful API in the next video